

SummaCham - PanelAMCHAM

Documentación Técnica Completa

Arquitectura, Apis, Queries, Plantillas y Gráficas
Explicado paso a paso para cualquier audiencia

Versión 4.1.0 | Febrero 2026 | Iustus Renidet

Que es SummaCham?

Es una aplicación de escritorio para gestionar presupuestos financieros.

Imagina un Excel inteligente que:

Se conecta al sistema contable (Aspel COI) para leer saldos reales

Permite que varios usuarios trabajen al mismo tiempo

**Tiene un flujo de aprobación:
cargar > revisar > aprobar**

Genera reportes y gráficas automáticamente

USUARIOS TÍPICOS:

Analistas: Cargan datos de presupuesto (RH, Finanzas, Eventos, etc.)

Supervisores: Revisan que los datos estén correctos

Directores: Aprueban los presupuestos finales

Administradores: Configuran usuarios, permisos y conexiones

COMO SE ACCEDE:

Local: <http://localhost:3005> (en la misma computadora)

Remoto: <https://panelamcham.iconetcloud.com.mx> (desde cualquier lugar)

Contenido de esta Presentación

PARTE 1

Arquitectura y Tecnologías

PARTE 2

Seguridad (Login, Cookies, Permisos)

PARTE 3

APIs - Que son, rutas reales con ejemplos

PARTE 4

APIs - Mapa completo de todas las rutas

PARTE 5

Queries - Como se calcula el Real

PARTE 6

Queries - Presupuesto y Acumulados

PARTE 7

Gestor de Plantillas

PARTE 8

Gráficas

PARTE 9

Base de Datos, Backups y Deployment

PARTE 1: Como esta construido SummaCham

El sistema tiene 3 grandes partes que se comunican entre si:



CÓMO FUNCIONA PASO A PASO:

1. El usuario abre la app (Electron abre una ventana con un navegador integrado)
2. El navegador carga `http://localhost:3005` (la página del sistema)
3. Cuando necesita datos, el frontend envía una PETICIÓN al backend
Ejemplo: "Dame el reporte RESUMEN de empresa1 para Junio 2025"
4. El backend recibe la petición y busca los datos en 2 lugares:
SQLite: Que cuentas mostrar, que fórmulas aplicar, que permisos tiene el usuario
Firebird: Los saldos reales del sistema contable Aspel COI
5. El backend responde con los datos en formato JSON
6. El frontend muestra los datos en tablas y gráficas

Tecnologías utilizadas

FRONTEND

Electron 39.2.7 App de escritorio (navegador integrado)
Bootstrap 5 Hace que se vea profesional
Chart.js 4.3 Gráficas interactivas (barras, líneas, pie)
DataTables 1.13 Tablas con búsqueda y filtros
SweetAlert2 Diálogos y notificaciones bonitas
ExcelJS / XLSX Exportar datos a Excel
jsPDF Exportar a PDF
Font Awesome 6.4 Iconos

BACKEND

Node.js Motor de JavaScript en servidor
Express.js Framework web (maneja peticiones)
Better-SQLite3 Base de datos local
node-firebird Conexión al sistema contable
bcryptjs Encriptar contraseñas
jsonwebtoken Tokens de acceso (JWT)
Joi Validar datos de entrada
Helmet Protección HTTP
express-session Sesiones de usuario

EN RESUMEN: El frontend usa HTML/JavaScript puro (sin React ni Angular).
El backend usa Node.js con Express para recibir peticiones y responder con datos.

PARTE 2: Como funciona el Login

Cuando escribes tu usuario y contraseña, pasa esto:

- 1 El frontend envía: `POST /api/auth/login` con `{ usuario: 'JPEREZ', contrasena: '...' }`
- 2 El servidor valida con Joi: usuario mínimo 2 caracteres, contraseña mínimo 6
- 3 Rate Limiting: Si hay 10+ intentos fallidos en 10 min, la IP se bloquea 5 minutos
- 4 Busca en SQLite: `SELECT * FROM usuarios WHERE usuario = 'JPEREZ'`
- 5 Compara contraseña con bcrypt (hash irreversible, nadie puede leerla)
- 6 Crea sesión + cookie 'panelamcham.sid' + JWT token (valido 1 hora)
- 7 Carga permisos del usuario (que empresas y módulos puede ver)

RESULTADO: El usuario entra al sistema con sus permisos cargados.

ARCHIVOS: `src/routes/auth.js` (login/logout) | `src/middleware/auth.js` (verificar sesión)

Cookies y Sesiones

Una cookie es un dato que el servidor manda al navegador.

Es como una PULSERA de acceso: la muestras cada vez que haces una petición.

Cookie de SummaCham:

Nombre: "panelamcham.sid"

httpOnly: true (JavaScript del navegador NO puede leerla - protege contra ataques)

secure: true (Solo funciona con HTTPS en producción)

maxAge: 30 min (Si no haces nada en 30 min, se cierra la sesión)

rolling: true (Se renueva con cada click, así no expira mientras trabajas)

Token JWT (va en el header de cada petición):

Duración: 1 hora (access token)

Refresh: 7 días (permite renovar el access token sin re-login)

Header: Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

COMO SE MANTIENE LA SESIÓN:

Login a las 09:00 -> expira 09:30

Click a las 09:15 -> se extiende a 09:45 (rolling)

Sin actividad 30 min -> sesión cerrada, debe re-loguearse

DONDE SE GUARDA: En SQLite (tabla de sesiones), NO en memoria del servidor

Sistema de Permisos

Cada usuario tiene permisos en 3 niveles:

NIVEL 1: USUARIO

es_admin_global (acceso total) | puede_agregar | puede_modificar | puede_eliminar

NIVEL 2: EMPRESA

Un usuario puede tener acceso a: empresa1 (CDMX), empresa2 (GDL), empresa3 (MTY), empresa4 (QRO)

NIVEL 3: MODULO + ACCION

Ejemplo: Juan Pérez en empresa1 (CDMX):

Modulo	Ver	Cargar	Revisar	Aprobar	
-----	-----	-----	-----	-----	
RH	SI	SI	NO	NO	(puede ver y cargar datos)
Finanzas	SI	NO	SI	NO	(puede ver y revisar)
RESUMEN	SI	NO	NO	NO	(solo lectura)

Tabla SQLite: permisos_modulo (usuario_id + empresa_id + módulo + permisos)

ADMIN GLOBAL: Salta todas las verificaciones, puede hacer todo en todo.

VERIFICACION: Cada petición al servidor se valida con requireAuth + tienePermisoModulo()

PARTE 3: Que es una API?

API = el MENSAJERO entre lo que ves en pantalla y donde están los datos.



Como funciona en SummaCham:

1. Haces click en "RESUMEN" en la app
2. El frontend automáticamente envía esta petición HTTP:

GET <http://localhost:3005/api/reportes/resumen?empresald=empresa1&anio=2025&mes=6>

3. La petición viaja con tu cookie de sesión y token JWT
4. El servidor la recibe, verifica que tengas permiso, y busca los datos
5. Responde con un JSON (datos estructurados)
6. El frontend pinta la tabla con esos datos

CADA PETICION LLEVA:

Cookie: panelamcham.sid=abc123... (identifica tu sesión)
Header: Authorization: Bearer eyJhbG... (tu token JWT)
Header: X-Empresa-Activa: empresa1 (empresa seleccionada)

Ejemplo real: Abrir el RESUMEN paso a paso

- 1 Abres RESUMEN.html en la app
- 2 resumen-view.js prepara la petición con empresa, año y mes
- 3 Envía: GET /api/reportes/resumen?empresald=empresa1&anio=2025&mes=6
- 4 src/server.js dirige /api/reportes/* a src/routes/reportes.js
- 5 middleware/auth.js verifica tu cookie y JWT -> carga tus permisos
- 6 reportes.js valida parámetros y llama a planeacionReportesEngine
- 7 Engine carga layout de SQLite: que cuentas y operaciones tiene RESUMEN
- 8 Engine consulta Firebird: saldos reales de SALDOS25 y presupuestos de PRESUP25
- 9 Engine calcula: Real, Presupuesto, Acumulados, Variaciones, Totales
- 10 Servidor responde JSON -> resumen-view.js pinta la tabla

ARCHIVOS INVOLUCRADOS:

Frontend: vistas/js/resumen-view.js -> Backend: src/routes/reportes.js

Motor: src/services/reportes/planeacionReportesEngine.js -> Firebird: planeacionCuentasService.js

APIs reales: Autenticación (/api/auth)

Estas son las rutas que manejan login, logout y renovación de sesión:

POST /api/auth/login

Que hace: Iniciar sesión

Envía: { "usuario": "JPEREZ", "contraseña": "MiClave123" }

Responde: { token: "eyJhbG...", refreshToken: "eyJhbG...",
 usuario: { id: 5, nombre: "Juan", esAdmin: false },
 permisos: { empresa1: { RH: { ver:true, cargar:true}, ... } } }

Errores: 400 (datos inválidos) | 401 (credenciales incorrectas) | 429 (bloqueado)

POST /api/auth/refresh

Que hace: Renovar el token sin re-login (cuando el access token expira)

Envía: { "refreshToken": "eyJhbG..." }

Responde: { token: "nuevoToken...", refreshToken: "nuevoRefresh..." }

POST /api/auth/logout

Que hace: Cerrar sesión (destruye cookie y sesión del servidor)

Responde: { mensaje: "Sesión cerrada" }

ARCHIVO: src/routes/auth.js

PROTECCION: Rate limiting (10 intentos / 10 min, bloqueo 5 min)

APIs reales: Reportes (/api/reportes)

Estas rutas generan los reportes financieros:

GET /api/reportes/summary?empresaId=empresa1&anio=2025&mes=6

Que hace: Genera el reporte SUMMARY (ingles) para empresa1, Jun 2025

Responde: { layout: [...], cuentas: [...], operaciones: [...], totales: {...} }

Archivo: src/routes/reportes.js -> summaryEngine -> planeacionReportesEngine

GET /api/reportes/resumen?empresaId=empresa1&anio=2025&mes=6

Que hace: Genera el reporte RESUMEN (español) para empresa1, Jun 2025

Responde: Misma estructura que summary pero con layout de RESUMEN

Archivo: src/routes/reportes.js -> planeacionReportesEngine.generarReporte()

GET /api/reportes/diagnostico-firebird?empresaId=empresa1

Que hace: Prueba si la conexión a Firebird funciona para esa empresa

Responde: { conectado: true/false, mensaje: "...", tiempoMs: 45 }

POST /api/reportes/operativo-excel-native

Que hace: Importa datos desde un archivo Excel al modulo operativo

Envía: Archivo Excel (.xlsx) como binario

PARÁMETROS COMUNES:

empresald = empresa1, empresa2, empresa3, empresa4 (cual empresa)

año = 2024, 2025, 2026 (que año fiscal)

mes = 1-12 (que mes quieres ver)

APIs reales: Presupuestos (/api/presupuestos)

GET /api/presupuestos?empresaId=empresa1&modulo=RH&anio=2025&capitulo=DEFAULT

Que hace: Trae los datos de presupuesto guardados para RH 2025

Responde: { datos: [...], estado: "cargado" }

GET /api/presupuestos/anios?empresaId=empresa1

Que hace: Lista los años disponibles en Firebird para esa empresa

Responde: { anios: [2023, 2024, 2025, 2026] }

GET /api/presupuestos/estado?empresaId=empresa1&modulo=RH&anio=2025

Que hace: Ver estado actual del presupuesto

Responde: { estado: "revisado", usuario: "JPEREZ", fecha: "2025-06-15" }

POST /api/presupuestos/estado

Que hace: Cambiar estado (ej: de "cargado" a "revisado")

Envía: { empresaId: "empresa1", modulo: "RH", anio: 2025,
nuevoEstado: "revisado" }

POST /api/presupuestos/guardar

Que hace: Guardar datos de presupuesto oficialmente

Envía: { empresaId: "empresa1", modulo: "RH", anio: 2025,
datos: [{ cuenta: "4100", ene: 100000, feb: 110000, ... }] }

GET /api/presupuestos/totales-capitulo?empresaId=empresa1&modulo=RH&anio=2025

Que hace: Totales resumidos por capitulo (CDMX, GDL, MTY, QRO)

APIs reales: Borradores (/api/borradores)

Los borradores guardan automáticamente el trabajo del usuario:

POST /api/borradores/guardar

Que hace: Guarda automáticamente mientras el usuario escribe (autoguardado)

Envía: { empresaId: "empresa1", modulo: "RH", anio: 2025,
capitulo: "DEFAULT", datos: [...] }

Nota: Se ejecuta automáticamente cada poco segundo

GET /api/borradores/listar?empresaId=empresa1&modulo=RH&anio=2025

Que hace: Lista todos los borradores guardados para ese modulo

Responde: [{ id: 12, fecha: "2025-06-15 10:30", estado: "borrador" }]

POST /api/borradores/enviar

Que hace: Marca el borrador como "enviado" (listo para revisión)

POST /api/borradores/revisar

Que hace: Un supervisor marca como "revisado"

POST /api/borradores/autorizar

Que hace: Un director da su aprobación final

POST /api/borradores/rechazar

Que hace: Regresa el borrador al estado anterior con comentarios

POST /api/borradores/recontabilizar

Que hace: Recalcula los saldos desde Firebird (refresca datos reales)

ARCHIVO: src/routes/borradores.js | FLUJO: borrador > enviado > revisado > autorizado

APIs reales: Usuarios (/api/usuarios)

GET /api/usuarios

Que hace: Lista todos los usuarios del sistema

Responde: [{ id: 1, usuario: "ICONET", nombre: "Admin", esAdmin: true },
{ id: 5, usuario: "JPEREZ", nombre: "Juan Pérez", esAdmin: false }]

GET /api/usuarios/5

Que hace: Datos de un usuario específico (por id)

Responde: { id: 5, usuario: "JPEREZ", nombres: "Juan Carlos", ... }

POST /api/usuarios

Que hace: Crear un usuario nuevo (requiere permiso puede_agregar)

Envía: { usuario: "MLOPEZ", nombres: "Maria", contraseña: "Segura123",
correo: "maria@empresa.com", es_admin_global: 0 }

PUT /api/usuarios/5

Que hace: Editar un usuario existente (requiere permiso puede_modificar)

Envía: { nombres: "Juan Carlos", correo: "nuevo@empresa.com" }

DELETE /api/usuarios/5

Que hace: Eliminar un usuario (requiere permiso puede_eliminar)

Nota: También elimina todos sus permisos (CASCADE)

POST /api/usuarios/5/permisos

Que hace: Asignar permisos a un usuario

Envía: { empresa_id: "empresa1", modulo: "RH",
puede_leer: 1, puede_cargar_guardar: 1, puede_revisar: 0 }

PARTE 4: Mapa completo de TODAS las APIs

AUTENTICACION Y USUARIOS

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/logout

GET|POST|PUT|DEL /api/usuarios

GET /api/empresas

REPORTES

GET /api/reportes/summary

GET /api/reportes/resumen

GET /api/saldos/:empresald

PRESUPUESTOS

GET /api/presupuestos

POST /api/presupuestos/guardar

GET|POST /api/presupuestos/estado

BORRADORES

POST /api/borradores/guardar

POST /api/borradores/enviar

POST /api/borradores/revisar

POST /api/borradores/autorizar

POST /api/borradores/recontabilizar

PLANTILLAS (LAYOUTS)

GET|POST /api/layouts-config/...

PUT|DEL /api/layouts-config/...

POST /api/insercion/...

GRÁFICAS Y CONFIG

GET|POST /api/graficas-config

GET|PUT /api/firebird-config

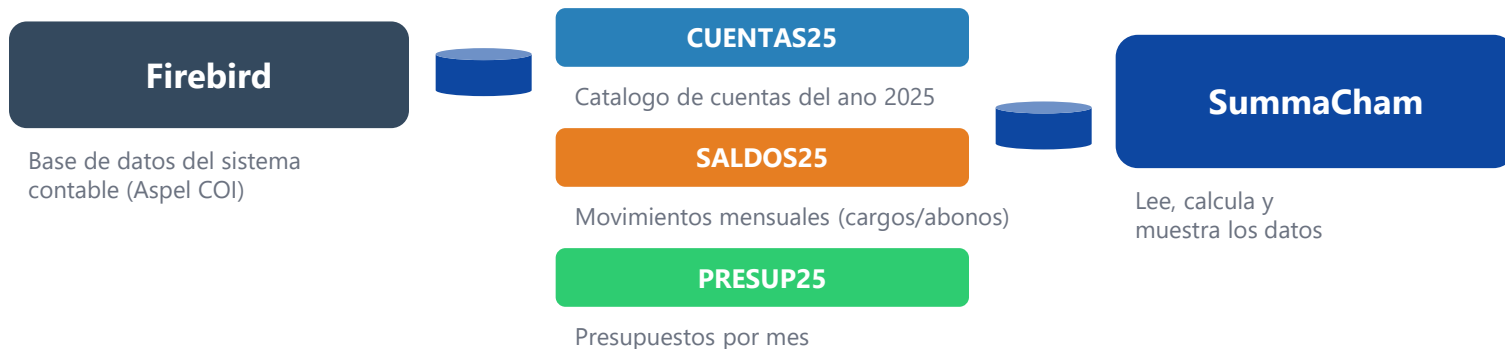
GET|POST /api/backups/...

GET|POST /api/notificaciones

GET|PUT /api/perfil

PARTE 5: ¿De dónde salen los números?

Los números que ves en pantalla vienen de Firebird (Aspel COI):



TABLAS POR AÑO: Firebird tiene tablas separadas para cada año fiscal:

2024: CUENTAS24, SALDOS24, PRESUP24

2025: CUENTAS25, SALDOS25, PRESUP25

2026: CUENTAS26, SALDOS26, PRESUP26

CADA CUENTA EN SALDOS TIENE:

12 columnas de CARGO (CARGO01...CARGO12) = dinero que ENTRA a la cuenta

12 columnas de ABONO (ABONO01...ABONO12) = dinero que SALE de la cuenta

QUE HACE SUMMACHAM CON ESTOS DATOS:

1. Lee las cuentas del layout (cuales mostrar)
2. Busca sus saldos en Firebird
3. Aplica la regla de NATURALEZA (siguiente slide)
4. Calcula Real, Presupuesto, Acumulados y Variaciones

Como se calcula el REAL (dato contable actual)

La NATURALEZA de la cuenta decide la fórmula:

Cuenta ACREEDORA (ingresos)
Cuentas que empiezan con 2, 3, 4

$$\text{Real} = \text{ABONOS} - \text{CARGOS}$$

Ejemplo cuenta 4100 "Membership" en Junio:

ABONO06 = 150,000 (créditos)

CARGO06 = 15,000 (débitos)

$$\text{REAL} = 150,000 - 15,000 = 135,000$$

Cuenta DEUDORA (gastos)
Cuentas que empiezan con 1, 5-9

$$\text{Real} = \text{CARGOS} - \text{ABONOS}$$

Ejemplo cuenta 5100 "Salaries" en Junio:

CARGO06 = 120,000 (débitos)

ABONO06 = 5,000 (créditos)

$$\text{REAL} = 120,000 - 5,000 = 115,000$$

¿POR QUÉ ES DIFERENTE?

Porque en contabilidad, un ingreso AUMENTA con abonos (créditos) y un gasto AUMENTA con cargos (débitos). La fórmula ajusta para que el numero siempre sea POSITIVO cuando la cuenta tiene saldo a favor.

ARCHIVO CLAVE: src/services/planeacionCuentasService.js

Función: obtenerSalDOSPorCuentas() y determinarNaturalezaReal()

PARTE 6: Como se calcula el PRESUPUESTO

El presupuesto es MÁS SIMPLE que el Real. Se lee directo:

```
-- Tabla Firebird: PRESUP25 (presupuestos del ano 2025)
-- Tiene 12 columnas: PRESUP01, PRESUP02, ... PRESUP12

SELECT pr.NUM_CTA,
       COALESCE(pr.PRESUP06, 0) AS presupuesto_junio
FROM PRESUP25 pr
WHERE pr.NUM_CTA IN ('4100', '5100')

-- Resultado:
-- 4100 (Membership):  PRESUP06 = 140,000  <- se usa TAL CUAL
-- 5100 (Salaries):    PRESUP06 = 110,000  <- se usa TAL CUAL
```

DIFERENCIA CLAVE CON EL REAL:

El REAL se CALCULA (abonos - cargos o cargos - abonos según naturaleza)

El PRESUPUESTO se LEE directamente de la tabla (sin calculo)

PARA QUE SIRVE LA COMPARACIÓN:

Si Real = 135,000 y Presupuesto = 140,000

Variación = $(135K - 140K) / 140K \times 100 = -3.6\%$

Significa que estás 3.6% por debajo de lo planeado

Como se calculan los ACUMULADOS (YTD)

YTD = Year-To-Date = Suma desde Enero hasta el mes que seleccionaste:

Si seleccionas MES = 6 (Junio):

Real Acumulado = Real Ene + Real Feb + Real Mar + Real Abr + Real May + Real Jun
Ppto Acumulado = PRESUP01 + PRESUP02 + PRESUP03 + PRESUP04 + PRESUP05 + PRESUP06
Real Acum AA = Misma suma pero del ano ANTERIOR (SALDOS24 en vez de SALDOS25)

EJEMPLO - Cuenta 4100 "Membership" - Selección: Junio 2025

Valor	Número	Qué significa
Real del Mes	135,000	Movimiento neto de Junio 2025
Presupuesto del Mes	140,000	Lo que se planeo para Junio 2025
Real Acumulado	810,000	Suma real de Enero a Junio 2025
Ppto Acumulado	840,000	Suma planeada de Enero a Junio 2025
Real Acumulado AA	780,000	Suma real de Enero a Junio 2024
Variación vs Plan	-3.6%	(Real - Ppto) / Ppto = debajo del plan

ESTOS 6 VALORES SON LOS QUE VES EN CADA FILA DEL REPORTE:

actualMonth (Real) | planMonth (Ppto) | prevMonth (Real año anterior)
actualYTD (Real Acum) | planYTD (Ppto Acum) | prevYTD (Real Acum AA)

ARCHIVOS: planeacionCuentasService.js (query SQL) + planeacionReportesEngine.js (calculo)

Flujo completo: De Firebird a tu pantalla

FIREBIRD - Aspel COI
SALDOS25 + PRESUP25 + CUENTAS25



planeacionCuentasService.js
obtenerSaldosPorCuentas(empresald, 2025, cuentas)

Ejecuta SQL, aplica naturaleza, calcula Real



planeacionReportesEngine.js
generarReporte('RESUMEN', empresa1, 2025, 6)

Carga layout SQLite, calcula operaciones, ordena



src/routes/reportes.js
GET /api/reportes/resumen

Valida permisos, responde JSON al frontend



resumen-view.js
Renderiza tabla HTML con los datos

Pinta filas, columnas, totales, colores

PARTE 7: Gestor de Plantillas

El Gestor de Plantillas define la ESTRUCTURA de los reportes.
Sin plantilla, no hay reporte. Es como el "molde" que dice que cuentas mostrar.

CUENTAS

Filas individuales. Ej: '4100' Membership
Se guardan en layout_cuentas (SQLite)

OPERACIONES

Filas de CÁLCULO. Ej: Total Income
Se guardan en layout_operaciones (SQLite)

SECCIONES

Agrupan cuentas. Ej: Income (CDMX)
Se guardan en layout_secciones (SQLite)

COMO SE USA (paso a paso):

1. Abres plantillas.html
2. Seleccionas: empresa + módulo + año + capítulo
3. El sistema carga el layout existente (o creas uno nuevo)
4. Agregas cuentas contables (número + nombre + sección)
5. Configuras operaciones (fórmulas que suman/restan secciones)
6. Ordenas con drag-and-drop (el orden en que se muestran)
7. Guardas -> listo para generar reportes

CADA LAYOUT ES ÚNICO POR:

Empresa (empresa1, empresa2...) + Módulo (RESUMEN, RH, Finanzas...)
Año (2024, 2025, 2026...) + Capítulo (CDMX, GDL, MTY, QRO, DEFAULT)

ARCHIVO PRINCIPAL: vistas/js/plantillas.js (~13,900 líneas)

APIs reales: Plantillas (/api/layouts-config)

BASE: /api/layouts-config ARCHIVO: src/routes/layoutRoutes.js

CARGAR Y GUARDAR:

GET	/.../RESUMEN/2025/DEFAULT	Cargar layout completo de RESUMEN 2025
POST	/.../RESUMEN/2025/DEFAULT	Guardar layout completo
POST	/.../RESUMEN/2025/DEFAULT/cuentas	Guardar solo las cuentas
POST	/.../RESUMEN/2025/operaciones	Guardar solo las operaciones

CONSULTAS:

GET	/.../RESUMEN/anios	Que años existen
GET	/.../RESUMEN/2025/capitulos	Que capítulos hay (CDMX, GDL, etc.)
GET	/.../RESUMEN/2025/estadisticas	Cuántas cuentas y operaciones tiene
GET	/.../RESUMEN/2025/existe	Verificar si ya existe este layout

EDICION INDIVIDUAL:

PUT	/.../RESUMEN/2025/DEFAULT/cuenta/4100	Editar cuenta 4100
DELETE	/.../RESUMEN/2025/DEFAULT/cuenta/4100	Eliminar cuenta 4100
PUT	/.../RESUMEN/2025/operacion/TOTAL_INCOME	Editar una operacion
DELETE	/.../RESUMEN/2025/operacion/TOTAL_INCOME	Eliminar una operacion

ADMINISTRACION:

POST	/.../RESUMEN/copiar	operación layout de otro año
POST	/.../RESUMEN/2025/DEFAULT/reordenar	Cambiar orden de filas
DELETE	/.../RESUMEN/2025	Eliminar layout completo
GET	/.../RESUMEN/2025/DEFAULT/bitacora	Ver registro de cambios

Como funcionan las Operaciones (fórmulas)

Las operaciones son filas que CALCULAN totales sumando o restando cuentas:

Ejemplo de fórmula guardada en layout_operaciones:

Operacion: "TOTAL INCOME"

Formula: [

```
{ operator: "+", type: "section", value: "Income (CDMX)" },  
{ operator: "+", type: "section", value: "Income (GDL)" },  
{ operator: "+", type: "section", value: "Income (MTY)" }
```

]

Resultado: TOTAL INCOME = Suma de TODOS los ingresos de todas las ciudades

Operacion: "NET INCOME"

Formula: [

```
{ operator: "+", type: "operation", value: "CONSOLIDATED INCOME" },  
{ operator: "-", type: "operation", value: "CONSOLIDATED EXPENSE" }
```

]

Resultado: NET INCOME = Ingresos totales - Gastos totales

TIPOS DE TÉRMINOS:

section: Suma TODAS las cuentas de esa sección

operation: Usa el resultado de OTRA operación (recursivo)

account: Usa el valor de UNA cuenta específica

ARCHIVO: src/services/reportes/planeacionReportesEngine.js (calcularTotalesOperacion)

PARTE 8: Sistema de Gráficas

Las gráficas toman los MISMOS datos del reporte y los visualizan:

Barra

Línea

Pie

Dona

Radar

Área

GRÁFICAS PREDEFINIDAS:

1. Resultado Operativo: Real Acum vs Ppto Acum vs Real AA por capitulo
2. Resumen Neto: Ganancia o perdida neta por capitulo
3. Consolidados: Ingreso total vs Gasto total
4. Tendencia mensual por ciudad
5. Gráficas personalizadas con Chart Wizard (4 pasos)

LOS 3 VALORES QUE SE GRAFICAN:

Real Acumulado (actualYTD)	Color azul oscuro #0d47a1
Ppto Acumulado (planYTD)	Color azul claro #60a5fa
Real Acum AA (prevYTD)	Color gris #94a3b8

SE GUARDAN EN: graficas_config (tabla SQLite)

APIs:

GET /api/graficas-config	Cargar configuración guardada
POST /api/graficas-config	Guardar configuración (solo admin)

ARCHIVOS: vistas/js/graficas-resumen.js + graficas-config.js + chart-wizard.js

PARTE 9: Base de Datos SQLite

Archivo local: datos/panel.sqlite - Guarda toda la configuración:

usuarios	Nombre, contraseña (hash), es_admin, permisos generales
permisos_modulo	Que puede hacer cada usuario en cada empresa/modulo
presupuestos_estado	Estado actual: sin-cargar, cargado, revisado, aprobado
presupuestos_guardados	Datos de presupuesto guardados oficialmente
PLAN Borradores	Autoguardado: datos que el usuario esta editando
layout_cuentas	Catalogo de cuentas por empresa/modulo/ano
layout_operaciones	Fórmulas de calculo (totales, consolidados)
layout_secciones	Agrupaciones (Income, Expense, etc.)
graficas_config	Configuración de gráficas guardadas
comentarios_celdas	Comentarios en celdas específicas del presupuesto
notificaciones	Alertas para los usuarios
sesiones	Sesiones activas (express-session store)

MODO: WAL (Write-Ahead Logging) permite lectura concurrente por múltiples usuarios

Sistema de Backups Automáticos

SummaCham copia automáticamente la base de datos para proteger contra pérdida:

1. Al iniciar la app, se hace el primer backup
2. Cada 60 minutos, se crea otro backup automáticamente
3. Se guardan los últimos 24 backups (los antiguos se eliminan)
4. Ubicación: datos/backups/panel.sqlite.bak_20260209_143022

APIs de Backups (/api/backups):

GET	/api/backups/status	Estado del servicio de backups
GET	/api/backups/list	Listar backups disponibles
POST	/api/backups/create	Crear backup manual (solo admin)
POST	/api/backups/restore	Restaurar desde un backup (solo admin)
GET	/api/backups/download/:fileName	Descargar un archivo de backup
PUT	/api/backups/config	Cambiar configuración (intervalo, etc.)

Archivo: src/routes/backups.js + src/services/backupService.js

CONFIGURACIÓN FIREBIRD (/api/firebird-config):

- | | | |
|------|----------------------------------|---|
| GET | /api/firebird-config | Ver config actual (host, port, empresas) |
| PUT | /api/firebird-config/base | Cambiar host, puerto, usuario, contraseña |
| PUT | /api/firebird-config/empresa/:id | Cambiar ruta de BD de una empresa |
| POST | /api/firebird-config/probar/:id | Probar conexión a una empresa |
- Solo administradores pueden modificar estas configuraciones.

Resumen de Seguridad

1

Contraseñas: bcrypt con 10 rondas de sal. NADIE puede leerlas.

2

Tokens JWT: Access token 1hr + Refresh token 7 días. Se renuevan automáticamente.

3

Sesiones: Cookie httpOnly (JS no puede leerla) + SQLite store.

4

Rate Limiting: Max 10 intentos/10 min. Bloqueo 5 min contra fuerza bruta.

5

Helmet: Cabeceras HTTP de seguridad automáticas (XSS, CSRF, etc.)

6

CORS: Solo orígenes autorizados pueden hacer peticiones.

7

Validación: Joi valida TODOS los datos de entrada antes de procesarlos.

8

SQL seguro: Queries parametrizados (?) previenen SQL injection.

9

Backups: Copias automáticas cada 60 min con checksum.

1

0

Auditoria: Se registra todo: logins, cambios, queries lentas.

Compilación y Distribución

COMO SE COMPILA:

npm install	Instalar dependencias
npm run dist	Crear instalador Windows (NSIS)
npm run build:portable	Crear versión portable (un solo .exe)

ARCHIVOS GENERADOS en ./dist/:

SummaCham Setup 4.1.0.exe	Instalador (requiere admin)
SummaCham 4.1.0.exe	Portable (sin instalación)
latest.yml	Metadatos para auto-update

ACTUALIZACION AUTOMÁTICA:

1. La app verifica en GitHub Releases si hay versión nueva
2. Si hay: pregunta "Descargar nueva versión?"
3. Descarga en segundo plano (no interrumpe el trabajo)
4. Al cerrar la app, instala automáticamente
5. Los DATOS del usuario se conservan (solo actualiza el programa)

SummaCham - PanelAMCHAM

Arquitectura en capas: Frontend + Backend + 2 bases de datos

APIs REST completas: 20+ grupos de endpoints documentados

Queries a Firebird con regla de naturaleza (Acreedora/Deudora)

6 valores por cuenta: Real, Ppto, AA, Real Acum, Ppto Acum, AA Acum

Gestor de Plantillas: 13,900 líneas que definen la estructura de reportes

Operaciones recursivas: fórmulas que se auto-construyen

Gráficas interactivas con Chart.js (barra, línea, pie, dona, radar)

Seguridad multicapa: bcrypt, JWT, rate limiting, helmet, CORS

Backups automáticos cada 60 minutos

Actualización automática vía GitHub Releases